

Multivariate Time-Series Anomaly Detection

A Complete Technical Textbook: Data, Models, Theory, and Four Rounds of Debugging

This document explains the entire project from raw data through the final model, in the order it actually happened - including the concepts behind every technique used, worked examples, and the reasoning behind every fix. It is meant to be readable on its own, without needing the code open alongside it.

Contents

Part I - Foundations

1. Introduction and Problem Statement
2. The Dataset
3. Exploratory Data Analysis and Sensor Selection

Part II - The Baseline Models

4. Isolation Forest: Theory and Application
5. The CNN Autoencoder: Theory and Application
6. The Hybrid Model: Fusion and Threshold Selection

Part III - Four Rounds of Debugging and Improvement

7. Round 1: Repository Cleanup and Critical Bug Fixes
8. Round 2: Checkpoint Selection and Overfitting
9. Round 3: Dilated Convolutions and Receptive Fields
10. Round 4: Mahalanobis Distance and F-beta Thresholds

Part IV - Results and Reflection

11. Final Results and Real-World Validation
12. Key Lessons and Future Directions

Part I

Foundations

1. Introduction and Problem Statement

This project detects anomalies in multivariate sensor time-series data from what appears to be an industrial process (the sensor tag names - PY23, FT703, PDI701, T701, LS701, etc. - follow standard instrumentation conventions: PY/PI for pressure, FT for flow, PDI for differential pressure, T for temperature, LS for level switch). Each raw file is one independent recording run of the process over time, with roughly 18-20 sensor channels sampled at each timestep.

The central constraint that shapes every modeling decision in this project: **there are no anomaly labels for training data**. Only a small validation set (10 runs) has ground truth; the real test set (53 runs) is completely unlabeled and scored externally by a competition portal. This rules out any ordinary supervised classifier - there is nothing to classify against during training. Every model here instead learns what *normal* behavior looks like, and flags departures from it.

The task: for every timestep in every test run, predict 0 (normal) or 1 (anomalous). Performance is measured by F1 score - the harmonic mean of precision (of everything flagged anomalous, how much really was) and recall (of everything truly anomalous, how much got caught).

2. The Dataset

Split	Runs	Labeled?	Purpose
train/	28	No	Model training (unsupervised)
val/ + val_labels/	10	Yes	Threshold tuning, checkpoint selection, all reported metrics
test/	53	No (scored externally)	Final competition submission

Each run is a CSV file with one row per timestep and one column per sensor. Runs vary in length (from a few thousand to tens of thousands of timesteps). Since test/ has no local labels, every F1 number reported anywhere in this project before actual portal submission is a **validation-set** estimate, not a true test score - a distinction maintained carefully throughout this document.

3. Exploratory Data Analysis and Sensor Selection

Before any modeling, the raw sensor set was analyzed to identify which of the ~20 channels actually carry useful information. Two independent analyses were run and then cross-checked against each other.

3.1 KDE-based distribution shape analysis

Kernel Density Estimation (KDE) was used to look at each sensor's overall value distribution shape, sorting sensors into four behavioral categories:

KDE Category	Sensors	Meaning
Highly Dynamic	FT703, PDI701, PDI702, FYI702	Wide-ranging, actively varying signal
Normal	T701-T712, PY23 (11 sensors)	Typical, moderately-varying distribution
Spike Sensitive	FT704	Distribution dominated by occasional spikes
Stable	LS701, LS702	Nearly constant - very low information content on its own, but reliable

3.2 Statistical sensor scoring

Separately, every sensor was scored on a composite of statistical properties computed per sensor over the whole training set: mean, standard deviation, skew, kurtosis, number of local peaks, number of unique values, average distribution shift over time, rolling-window mean/std stability, and a trend-strength measure. These were combined into a weighted `sensor_score` (see `src/sensor_selector.py` / `src/core/feature_selection.py`), sorting sensors into three final usefulness tiers:

Final Category	Sensors	Count
Highly Useful Sensor	LS701, LS702, FT703, PDI701, PDI702, FT704	6
Moderate Sensor	T702, T703, T708, T711, T712, T705	6
Unstable / Low Utility Sensor	FYI702, T701, T704, T706, T709, PY23	6

Notably, the two independent methods largely agree but not perfectly - the sensors LS701/LS702 (KDE: "Stable", almost constant) still scored as "Highly Useful" statistically, because a sensor that's normally rock-steady is extremely informative *precisely when it moves* - a concept that becomes directly relevant again in Round 4's Mahalanobis scoring (Chapter 10). After correlation-based filtering (dropping sensors with >0.85 correlation to another already-kept sensor), **16 sensors** were kept for modeling (see `data/processed/selected_sensors.json`).

Part II

The Baseline Models

4. Isolation Forest: Theory and Application

Isolation Forest is a statistical outlier detector with no concept of time at all - it looks at a single snapshot of features and asks a purely geometric question.

4.1 The core idea

Build many random decision trees. At each node, pick a random feature and a random split value between its min and max. Keep splitting until every point is alone in its own leaf. **The number of splits it took to isolate a point is its anomaly score** - fewer splits needed means more anomalous.

Isolation Forest: outliers separate from the crowd in far fewer random splits

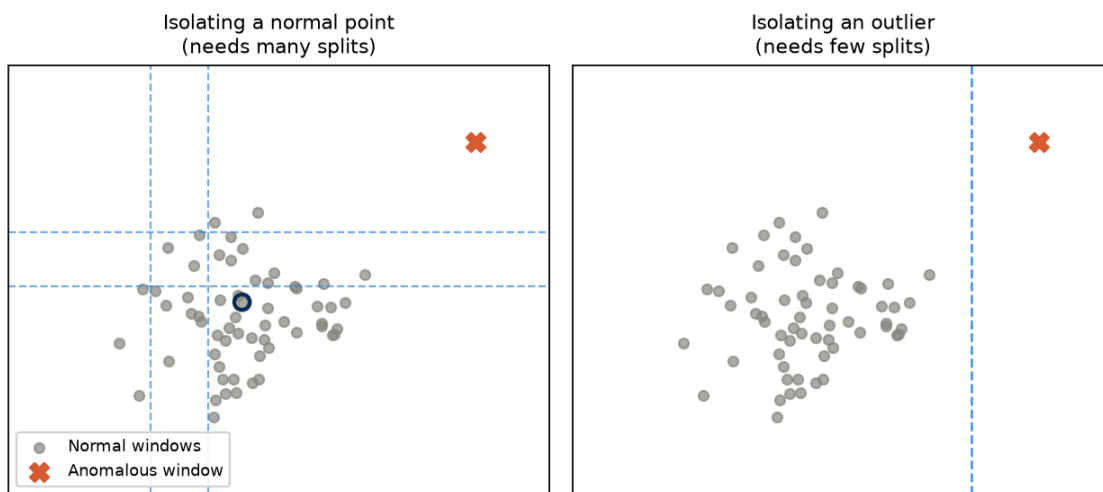


Figure 4.1 - A point buried in a dense cluster needs many random splits to isolate; an outlier separates from the crowd almost immediately.

Intuition: imagine 60 normal windows clustered together and one wildly different window. A random split line has a good chance of immediately separating the outlier from everyone else, since it sits far from the crowd in some direction. A normal point deep inside the cluster needs many splits before it's alone, because every random cut is likely to still leave it grouped with several neighbors.

4.2 Features used in this project

Two different feature representations of Isolation Forest exist in this project:

- **Tabular baseline** (src/pipelines/run_isolation_forest.py): lag, delta, rolling mean/std, EWMA, and pairwise sensor-interaction features computed per row (src/core/feature_pipeline.py).

- **Window-based** (used inside the hybrid, `src/models/isolation_forest/features.py`): 7 summary statistics (mean, std, min, max, median, 25th/75th percentile) per sensor per 200-timestep window.

This project's Isolation Forest is trained only on data assumed normal (the unlabeled training set), and at prediction time, higher isolation-based score means more anomalous. Standalone validation performance settled around $F1 = 0.49$ (window-based) and $F1 = 0.44$ (tabular) - a real, if modest, signal.

5. The CNN Autoencoder: Theory and Application

The second detector learns actual temporal shape, using a completely different mechanism: reconstruction.

5.1 The core idea

An autoencoder is two networks glued together: an **encoder** that compresses an input down to a small summary (a "bottleneck"), and a **decoder** that tries to rebuild the original input from just that summary. Training forces the model to keep only whatever information is most useful for reconstruction, and throw the rest away.

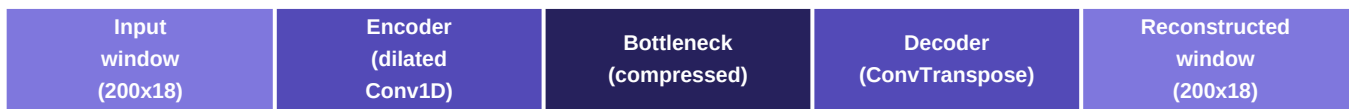


Figure 5.1 - The autoencoder pipeline for one window.

Trained only on windows assumed normal, with no anomaly labels used at all: the loss function simply penalizes the difference between the input and its own reconstruction. Because it never practices on anomalies, it becomes specifically good at reconstructing normal patterns - and when handed something genuinely unusual, its best guess (still built from a "normal" vocabulary) misses badly.

Worked example: a normal window has temperature gently oscillating 20.1-20.4 and vibration steady near 0.50. The model learns this pattern well. Given an anomalous window where vibration has spiked and stuck at 3.8, the model's reconstruction - built only from its "vibration hovers near 0.5" vocabulary - comes out close to 0.5, wildly wrong versus the actual 3.8. That gap is the anomaly signal.

5.2 Reconstruction error as the anomaly score

For each timestep and sensor, compute the squared difference between the true value and the reconstructed value. How these per-sensor, per-timestep numbers get collapsed into one anomaly score per window turned out to be one of the most consequential decisions in the whole project - covered in depth in Chapter 10.

6. The Hybrid Model: Fusion and Threshold Selection

The premise: the CNN catches slow temporal drift; Isolation Forest catches sudden statistical outliers independent of temporal shape. Combining them should, in principle, cover more ground than either alone.



Figure 6.1 - The hybrid fusion pipeline.

Both branches' scores are min-max normalized to [0,1] using validation-set statistics (never test-set statistics, to avoid leaking information from data being predicted on), then combined as a weighted average: $\text{Score} = \alpha * \text{CNN_score} + (1 - \alpha) * \text{IF_score}$. A single threshold then turns the continuous fused score into a binary 0/1 prediction. Both the fusion weight α and the threshold are chosen by testing candidates against the labeled validation set and keeping whichever maximizes the chosen metric.

Part III

Four Rounds of Debugging and Improvement

The baseline hybrid model, as originally built, reported $F1 = 0.40$ (Isolation Forest), 0.60 (CNN), 0.62 (Hybrid) - but these numbers came from an implementation that lived only in notebooks, copy-pasted with drift, and measured at coarse *window* granularity rather than the per-timestep granularity the real submission actually needs. Four rounds of work followed, each addressing a different layer of the problem.

7. Round 1: Repository Cleanup and Critical Bug Fixes

Before any modeling changes, six concrete bugs were found and fixed, each with a real conceptual lesson behind it.

Isolation Forest features flattened. Window statistics were averaged across *all 18 sensors at once* into one scalar per statistic, destroying the information about which sensor was actually off. *Fixed to keep one value per sensor.*

Threshold off-by-one. A library function's output array (`precision_recall_curve`) was indexed one slot wrong, so the applied threshold never matched the reported "best" $F1$. *A pure indexing bug - the displayed number was simply incorrect.*

Windows crossed run boundaries. All files in a split were concatenated before slicing into windows, so a window could contain one run's ending glued to an unrelated run's beginning. *This is a data-leakage/contamination concept: supposedly clean examples weren't clean.*

CNN epoch count regressed. 50 epochs had already been proven necessary by earlier tuning, but the production pipeline had silently reverted to 10. *Configuration drift - a validated setting was lost.*

Coarse scoring granularity. The submission format needs a prediction per timestep, but scoring produced one label per window, blindly broadcast across ~ 10 raw timesteps. *Fixed to score densely (every timestep, via overlapping windows averaged together) - a granularity mismatch between training/eval and the actual prediction target.*

Hardcoded fusion weight. 0.85 CNN / 0.15 Isolation Forest, picked once by hand. *Replaced with a sweep against validation data.*

Result: validation $F1$ settled at CNN 0.408 , Isolation Forest 0.491 , Hybrid 0.494 (per-timestep, a stricter metric than the baseline's window-level numbers). Submitted to the real competition portal: public $F1 = 0.399$, private $F1 = 0.471$ - close to the validation estimate, confirming the evaluation methodology itself was trustworthy even though the models still had room to improve.

8. Round 2: Checkpoint Selection and Overfitting

8.1 The overfitting concept

Watching the CNN train for 50 epochs, the training loss fell smoothly the *entire time* with no plateau. This is a classic overfitting signature: the model kept getting better at reconstructing its training data specifically - including, plausibly, some real anomalies hiding in the unlabeled training set (nothing guarantees train data is anomaly-free). The more thoroughly it learns to reconstruct everything it's shown, including anomalies, the smaller the gap between normal and anomalous reconstruction error becomes - directly eroding the signal the whole approach depends on.

Fix: evaluate on the labeled validation set every 5 epochs during training, and keep whichever epoch's weights scored best - instead of blindly using the final epoch. This is standard early stopping, applied as post-hoc best-checkpoint selection (see `eval_every/eval_fn` in `src/models/cnn/train.py`).

Result: CNN standalone F1 jumped from 0.408 to 0.458 (best checkpoint at epoch 5, not epoch 50), directly confirming the overfitting diagnosis. A side experiment also swept Isolation Forest's contamination hyperparameter and found it had *exactly zero effect* on the reported score - because that parameter only shifts the model's own internal classification cutoff, and this pipeline scores with the continuous decision function and does its own separate threshold search, never touching that internal cutoff at all. Hybrid F1 stayed flat at 0.494 despite the CNN's real improvement - the first hint that something structural, not just the model, was capping the result.

9. Round 3: Dilated Convolutions and Receptive Fields

9.1 The receptive field concept

A convolutional layer's "receptive field" is how much of the input it can actually take into account when producing one output value. The original architecture (a single Conv1d block, kernel size 3, no dilation) had an effective receptive field of only about 3 timesteps - out of a 200-timestep window.

This mattered because the project's own earlier tuning experiments had already proven the anomalies here are **slow, sustained drifts**, not sharp spikes - F1 kept improving as window size grew from 20 to 200 timesteps. A model that can only "see" 3 timesteps at a time cannot actually exploit a 200-step window, no matter how large the window is made.

Dilated convolutions fix this by making each layer skip over gaps between the points it examines (dilation rates 1, 2, 4, 8 stacked), expanding the effective receptive field exponentially rather than linearly, without enlarging the window or the memory footprint.

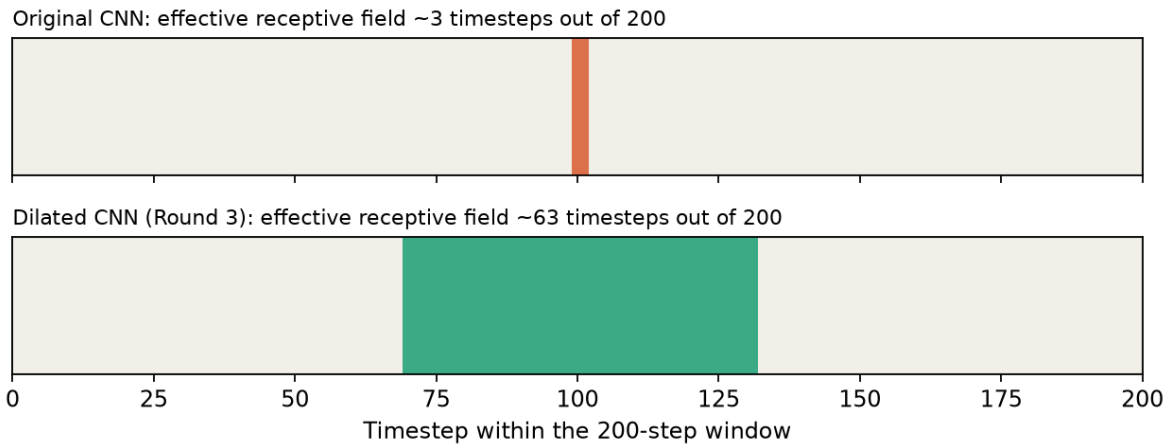


Figure 9.1 - The dilated architecture (Round 3) sees roughly 20x more of the window per output value than the original single-block CNN.

LSTM and Transformer architectures were considered and set aside: LSTMs are slower and less stable to train on 200-step sequences; Transformers are data-hungry and add more tuning risk than justified for this dataset's size. The dilated CNN was the closest, lowest-risk extension of the existing model.

Result: CNN standalone F1 improved again, 0.458 to 0.484. **But hybrid F1 was still stuck at 0.494** - two consecutive rounds of real, verified CNN improvement with zero movement in the number that actually mattered. A pattern, not a coincidence.

10. Round 4: Mahalanobis Distance and F-beta Thresholds

Rounds 2 and 3 both improved the *model*. Round 4 asked a different question: was the model's output even being turned into a decision correctly? This round was inspired partly by re-reading the original 2016 EncDec-AD paper on LSTM autoencoders for anomaly detection, which uses exactly these two techniques.

10.1 Mahalanobis distance: covariance-aware scoring

The CNN's reconstruction error was being averaged across *every sensor and every timestep* into one scalar. The problem: some sensors are naturally noisy even when everything is fine; others are naturally razor-precise. A genuine spike on a precise sensor, averaged together with a noisy sensor's ordinary wobble, produces an unremarkable-looking number - the real signal gets buried in the average.

Same reconstruction errors, two different scoring methods

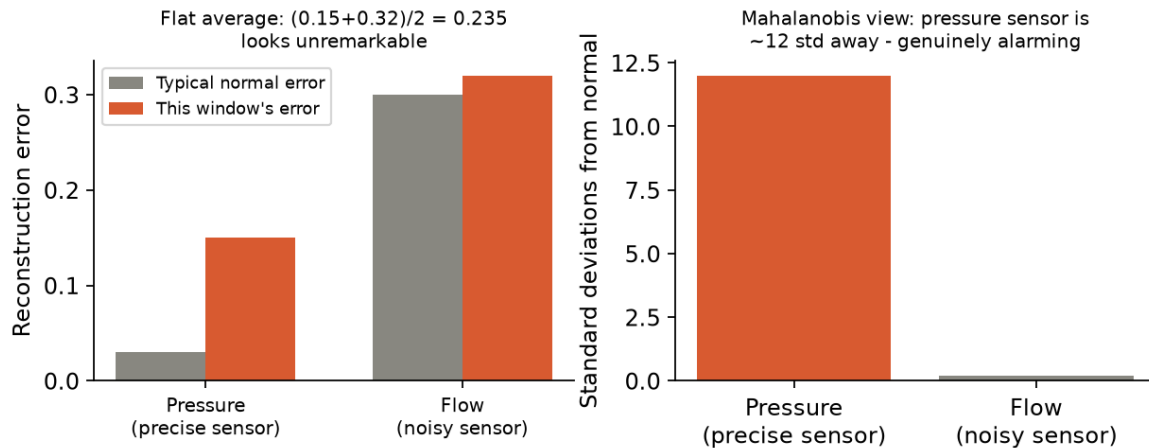


Figure 10.1 - Same two reconstruction errors, scored two different ways. Flat averaging hides the pressure sensor's alarming deviation; Mahalanobis distance reveals it.

Fix: keep the error *per sensor* instead of one flat average. Using only confirmed-normal validation windows, learn each sensor's typical error mean and spread, and how sensors' errors normally co-vary (their covariance). Score a new window by how many "standard deviations of surprise" its error vector represents, in each sensor's own natural terms, accounting for cross-sensor correlation too. This is precisely what a Mahalanobis distance computes: $\text{score} = (\text{error} - \text{mean})^T \cdot \text{covariance}^{-1} \cdot (\text{error} - \text{mean})$. Implemented via `sklearn.covariance.LedoitWolf`, a shrinkage estimator that stays numerically stable even with a modest number of samples relative to 18 sensor dimensions (see `src/evaluation/metrics.py::fit_mahalanobis`).

10.2 F-beta thresholds: choosing where on the precision/recall dial to sit

Picture the anomaly threshold as a dial. Turned low: catch nearly every real anomaly (high recall), but flag enormous numbers of normal windows too (low precision) - like a smoke alarm that trips from making toast. Turned high: almost no false alarms (high precision), but miss real anomalies (low recall). Optimizing for plain F1 treats both kinds of mistake as equally costly - and given how overlapped the normal/anomalous score distributions had been all project, the F1-optimal point kept landing on the aggressive, high-recall side: 65-88% of all timesteps flagged anomalous, against a true rate of 24%.

Fix: switch to F-beta with $\beta = 0.5$, which weights precision twice as heavily as recall when choosing the "best" threshold, deliberately pushing the dial toward "only flag when confident."

10.3 Results, and one more grid-boundary bug

Metric	Round 3	Round 4
CNN ROC-AUC (separability)	0.605	0.768
CNN F1	0.484	0.581
CNN anomaly rate (true = 0.24)	0.894	0.224

Both fixes worked together: separability jumped (a threshold-independent measure, so this improvement is not just an artifact of picking a different cutoff), and calibration became nearly exact at the same time. But once the CNN got this strong, the fusion weight sweep - which only tested CNN weights up to 0.9 - kept selecting that 0.9 boundary, the classic sign a true optimum lies outside the tested range. Adding 1.0 ("pure CNN, no fusion") to the grid let the sweep discover that not fusing at all is currently the best move: the hybrid's selected weight is exactly 1.0, meaning Isolation Forest contributes nothing to the winning configuration right now.

Part IV

Results and Reflection

11. Final Results and Real-World Validation

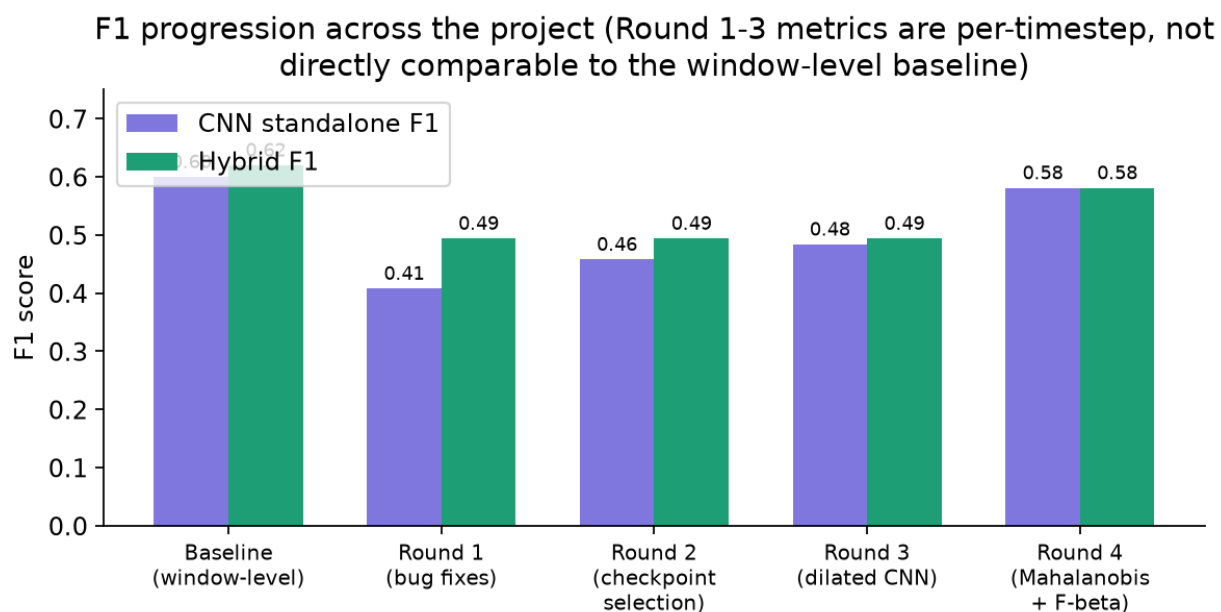


Figure 11.1 - F1 across the whole project. Three rounds of real model improvement (2, 3) left the hybrid number completely flat; Round 4 - fixing the scoring formula and the decision rule, not the model - is what finally moved it.

The final model achieved $F1 = 0.581$ on the validation set, and **$F1 = 0.59$ on the real competition portal** - a validation number that was, if anything, slightly conservative relative to the true test result, confirming the evaluation methodology built up over four rounds is genuinely trustworthy.

This is up from the first real portal submission of public $F1 = 0.399$ / private $F1 = 0.471$, and up from a validation $F1$ that sat at exactly 0.494 through three separate rounds of work before Round 4 broke through it.

12. Key Lessons and Future Directions

12.1 The throughline of this project

Rounds 2 and 3 made the underlying model measurably smarter, and both improvements held up under scrutiny - yet neither one moved the metric that mattered. The reason: the model's output still had to pass through a scoring formula (flat averaging across sensors) and a decision rule (plain- $F1$ threshold selection) that were both quietly discarding information and mis-calibrating the result. No amount of model improvement can show through a bottleneck in how that improvement gets measured and turned into a decision. Round 4 fixed the bottleneck itself, not the model - and that is what finally let two rounds of real,

previously invisible CNN progress show up in the final number.

12.2 What's still open

- The "hybrid" model is currently, honestly, just the CNN - Isolation Forest's optimal weight is 0. Worth deciding whether to present this plainly, or invest in richer Isolation Forest features (cross-sensor interaction terms, frequency-domain features) so it can meaningfully rejoin the ensemble.
- Training-set contamination remains unaddressed directly - train data is not guaranteed anomaly-free, and a training procedure that iteratively down-weights the highest-error windows could plausibly help both branches at once.
- The validation set is small (10 runs); every threshold, architecture, and hyperparameter choice in this project has been tuned against it. A k-fold or held-out tuning split would give a more independent read on generalization before further changes.